

UNIT I: **OOPs CONCEPT**

DR. MRS. V. V. HANCHATE

UNIT I: OOPs CONCEPT

Inheritance Types Of Inheritance: Introduction, Classes Inheritance, Interface Inheritance, Multilevel Inheritance, Accessing members of other class/interface, Chaining Constructor using this() and Super(), This() constructor call, Super()constructor call, Dynamic method Dispatch, Method Overriding, Modifiers: Java Access Specifies: Public, Protected, Private, Default Other modifiers for members: Static, Final, Abstract, Synchronized, Native, Transient, Volatile, Other modifiers for classes: Abstract classes, Final classes.

COURSE OBJECTIVE

- **Discuss** the concept of inheritance and dynamic binding using Java

COURSE OUTCOME

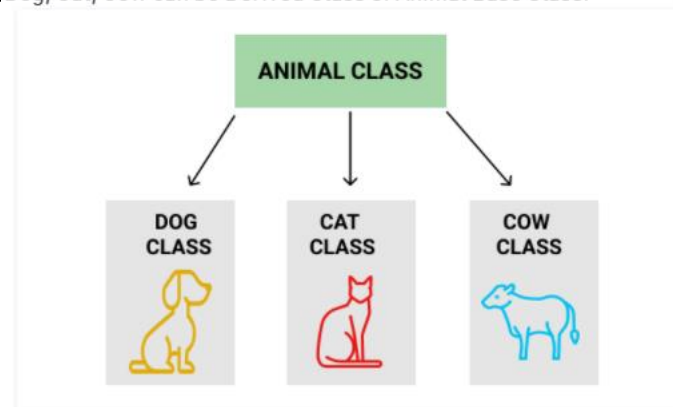
- **Illustrate** the concept of inheritance and dynamic binding using Java.
(Bloom's Level 2 : Understand)

Modern College of Engineering

★ Pune - 5 ★

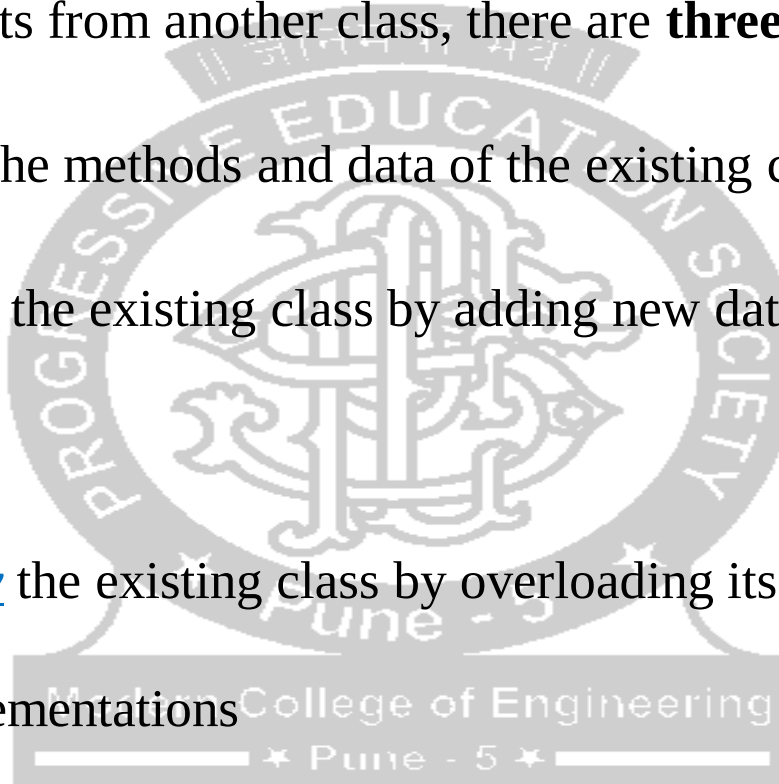
INHERITANCE

- Allows programmers to create new classes based on an existing class
- Methods and attributes from the parent class are inherited by the newly-created class
- New methods and attributes can be created in the new class, but don't affect the parent class's definition.
- The capability of a class to derive properties and characteristics from another class is called Inheritance. Inheritance is one of the most important features of Object-Oriented Programming.
- The old class is referred to as the **Base Class** & the new one is called the **Derived Class** or **Subclass**. Dog, Cat, Cow can be Derived Class of Animal Base Class.



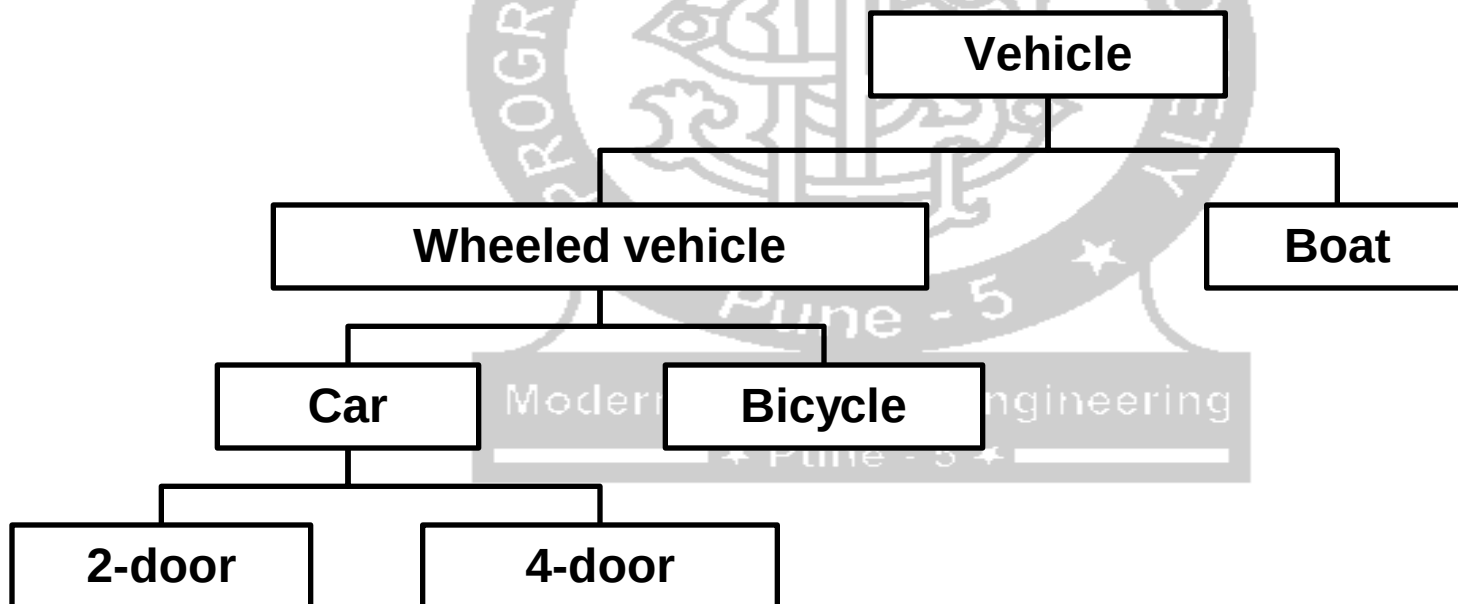
ADVANTAGES OF INHERITANCE

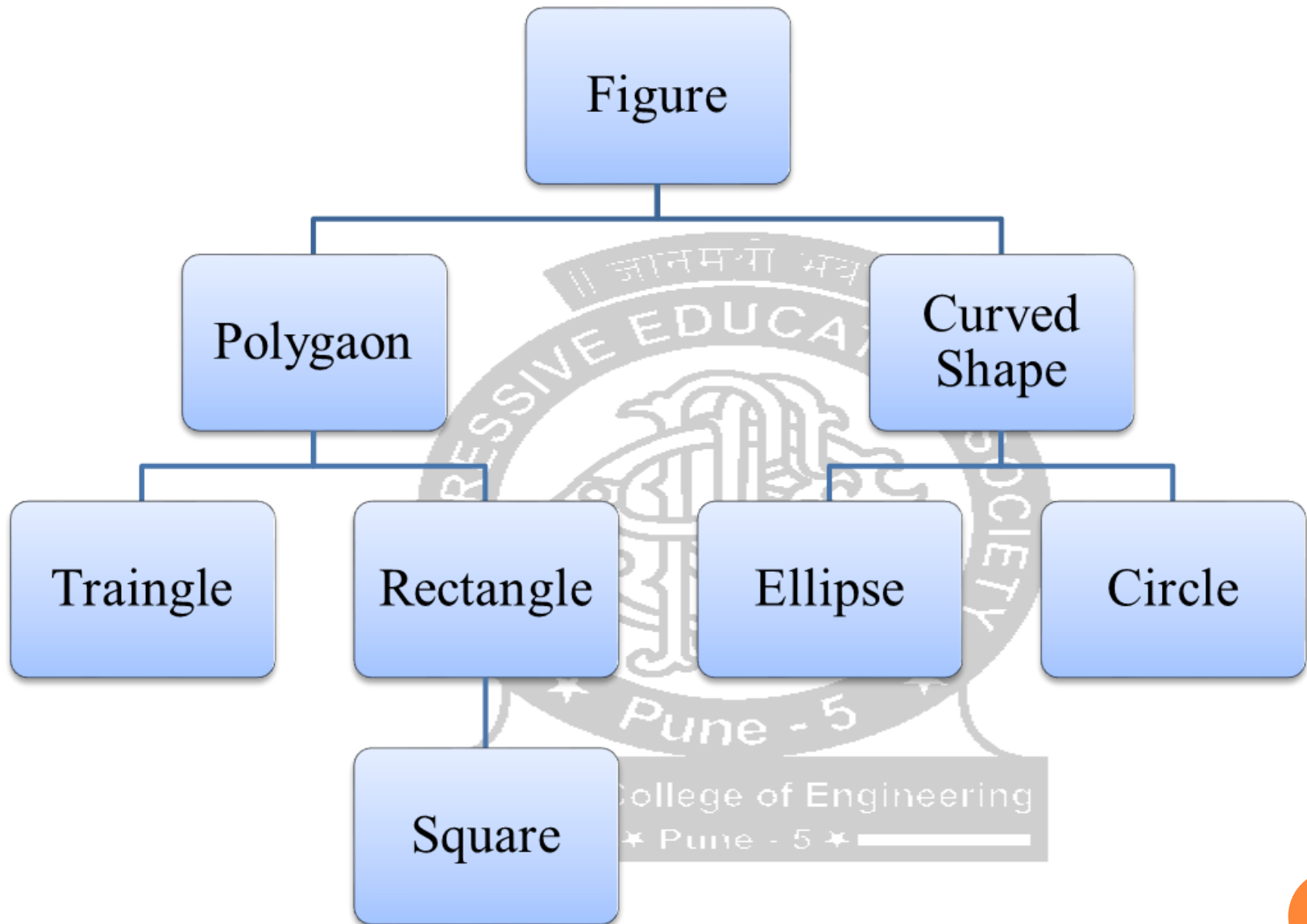
- When a class inherits from another class, there are **three** benefits:
 - You can reuse the methods and data of the existing class
 - You can extend the existing class by adding new data and new methods
 - You can modify the existing class by overloading its methods with your own implementations



ARRANGE CONCEPTS INTO AN INHERITANCE HIERARCHY

- Concepts at higher levels are more general
- Concepts at lower levels are more specific (inherit properties of concepts at higher levels)



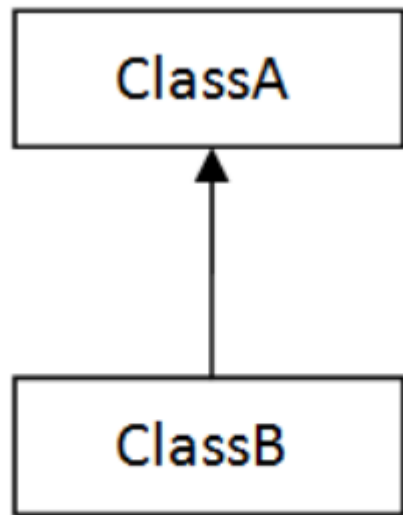


TYPES OF INHERITANCE

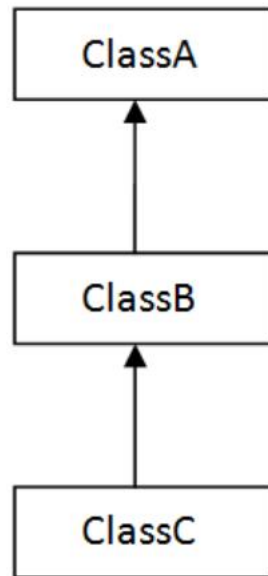
There are three types of inheritances in java

1. Single inheritance
2. Multilevel inheritance
3. Hierarchical inheritance

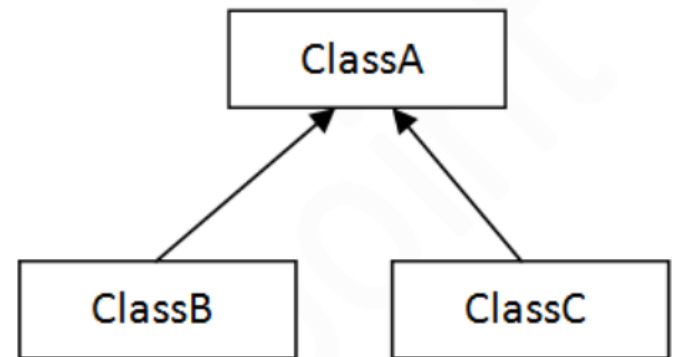
In java programming, multiple and hybrid inheritance is supported through interface only. We will learn about interfaces later.



1) Single



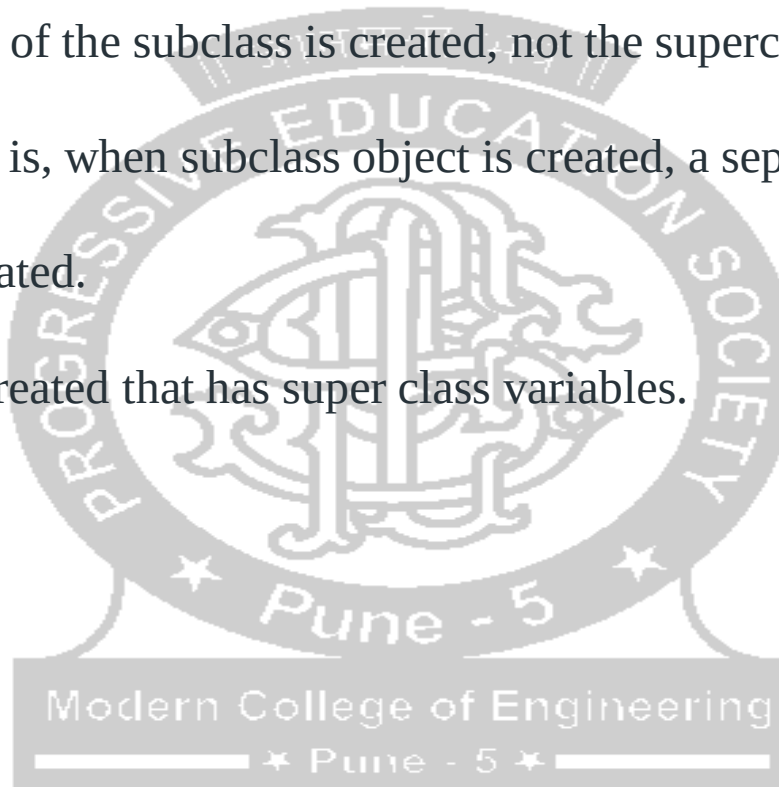
2) Multilevel



3) Hierarchical

POINTS TO REMEMBER:

- Object of the subclass we can also access the members of a superclass.
- Inheritance only the object of the subclass is created, not the superclass.
- An important point to note is, when subclass object is created, a separate object of superclass object will not be created.
- Only a subclass object is created that has super class variables.



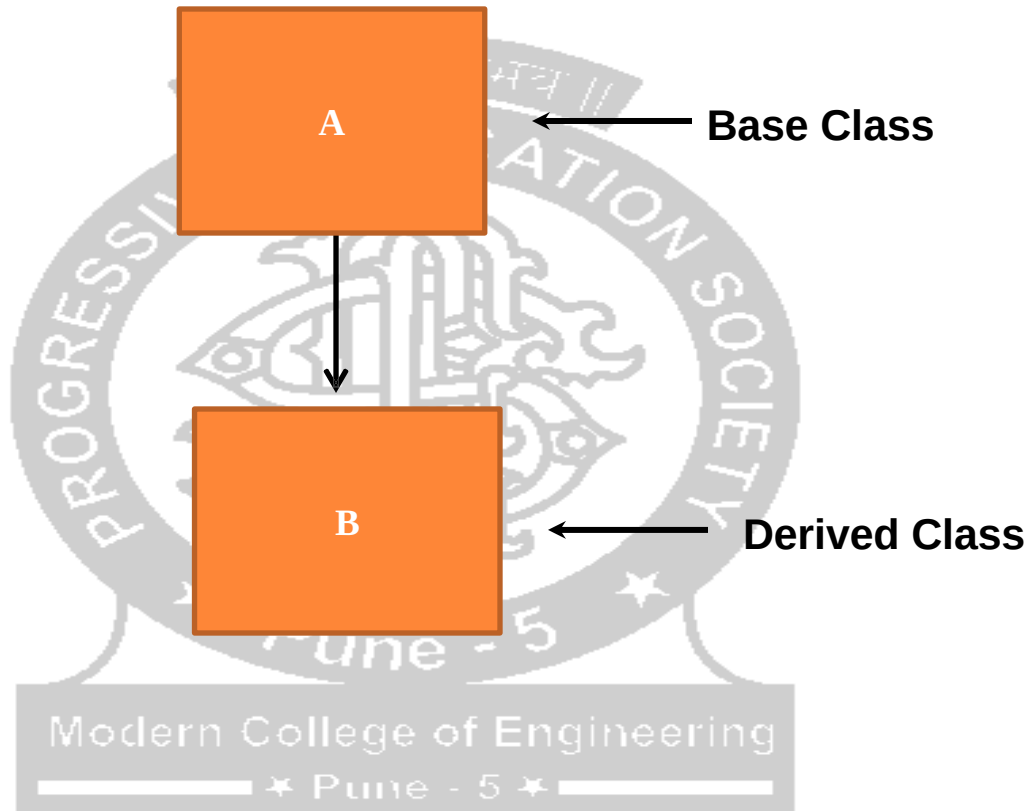
INHERITANCE AND MEMBER ACCESSIBILITY

- A member of the class is said to be accessible when access to it is allowed.
- Declaration of a member in a class allow control over member access.
- The accessibility of a member is specified by including access modifiers in its declaration.

Modifier	Class	Package	Subclass	Remark
Public	Yes	Yes	Yes	Access is not limited
Protected	Yes	Yes	Yes	Access is limited to its class, package and those derived from the class
No modifier	Yes	Yes	No	Access is limited to its class and package and subclass within the package
Private	Yes	No	No	Access is limited to its class.

- A derived class has access to all public and protected members of base class with both classes not necessarily in same package.
- Private members are also inherited by derived class but are not directly accessible.
- The accessible members can be used , replaced, hidden with new members.

SINGLE INHERITANCE



EXAMPLE OF SINGLE INHERITANCE

```
class Calculus
{
    public int add(int x,int y)
    {
        int z = x+y;
        return z;
    }

    public int sub(int p,int q)
    {
        int k = p-q;
        return k;
    }
}
```

```
class My_calculus extends Calculus
{
    public int mul(int f,int g)
    {
        int j = f*g;
        return j;
    }
}
```

```
public class Inheritance_calculator {

    public static void main(String[] args)
    {
        My_calculus obj1 = new My_calculus();
        int a= obj1.add(4,5);
        int b = obj1.sub(10,8);
        int c = obj1.mul(2,3);
        System.out.print("\n" + "Addition is: "+a);
        System.out.print("\n" + "Subtraction is: "+b);
        System.out.print("\n" + "Multiplication is: "+c);
    }
}
```

Note: Object is to be created for derived or child class only.
Object of parent class need not be created separately.

CONSTRUCTORS IN INHERITANCE

When an object of child class is created it will call the constructor in parent class first and then call the constructor of child class. This type of call is called implicit call.

Ex:

```
class A
{
int a;
public A()
{
System.out.print(" This is class A constructor");
}
}
class B extends A
{
public B()
{ int b;
System.out.print(" This is class B constructor");
}
}
public class Constructor_inheritance {

public static void main(String[] args) {
B obj = new B();
}
}
```

Note: In this program class B constructor will call class A constructor first

If there is no constructor in base class then compiler will define it and that constructor is called default constructor i.e with no argument and no statements
Object obj will have memory for both variables a and b.

Output :

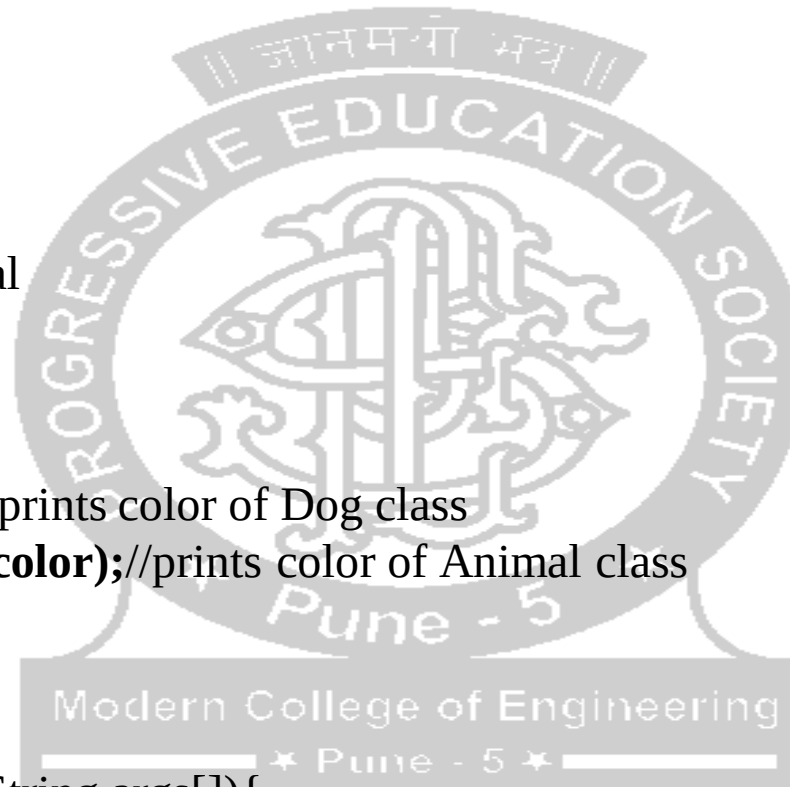
This is class A constructor
This is class B constructor



USING SUPER KEYWORD

This keyword is used to access the variables, methods or constructor of the base or parent class. We can use super keyword to access the **data member or field of parent class**. It is used if parent class and child class have same fields.

```
class Animal
{
    String color="white";
}
class Dog extends Animal
{
    String color="black";
    void print(){
        System.out.print(color);//prints color of Dog class
        System.out.print(super.color);//prints color of Animal class
    }
}
class TestSuper{
    public static void main(String args[]){
        Dog d=new Dog();
        d.print ();
    }
}
```



USING SUPER KEYWORD :SUPER CAN BE USED TO INVOKE PARENT CLASS METHOD

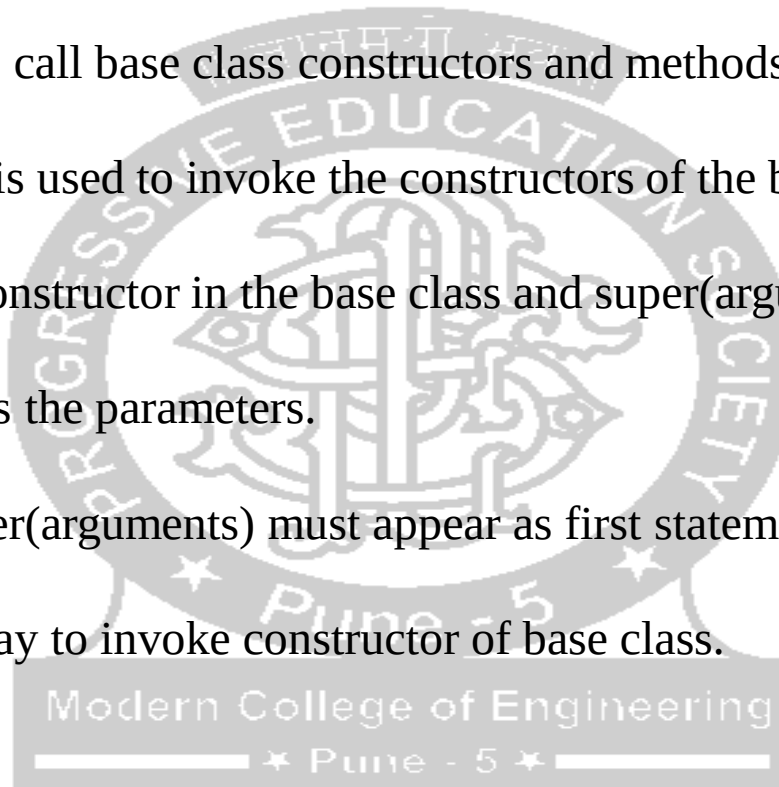
The super keyword can also be used to invoke parent class method. It should be used if subclass contains the same method as parent class. In other words, it is used if method is overridden.

```
class Animal{  
void eat()  
{  
System.out.print ("eating...");  
}  
}  
class Dog extends Animal{  
void eat()  
{  
System.out.print ("eating bread...");  
}  
void bark()  
{  
System.out.print ("barking...");  
}
```

```
void work()  
{  
super.eat();  
bark();  
}  
class TestSuper2{  
public static void main(String args[]){  
Dog d=new Dog();  
d.work();  
}}
```

USING THE KEYWORD SUPER :SUPER IS USED TO INVOKE PARENT CLASS CONSTRUCTOR.

- ❖ Explicit call to constructor can be done using super keyword.
- ❖ The keyword super is used to call base class constructors and methods.
- ❖ Super() or super(arguments) is used to invoke the constructors of the base class.
- ❖ super() invokes the default constructor in the base class and super(arguments) invokes the base class constructor that matches the parameters.
- ❖ The statement super() or super(arguments) must appear as first statement of the derived class constructor and is the only way to invoke constructor of base class.



PROGRAM TO EXPLAIN USE OF SUPER TO INVOKE PARENT CLASS PARAMETERIZED CONSTRUCTOR

```
class A
{
    int a;
    A(int a)
    {
        this.a = a;
        System.out.print("I am in class A:");
        System.out.print("\n"+a);
    }
}

class B extends A
{
    int b;
    B()
    {
        super(10);
        System.out.print("\n"+"I am in class B:");
    }
}
```

```
public class Super_constructor {

    public static void main(String[] args) {
        B obj = new B();
    }
}
```

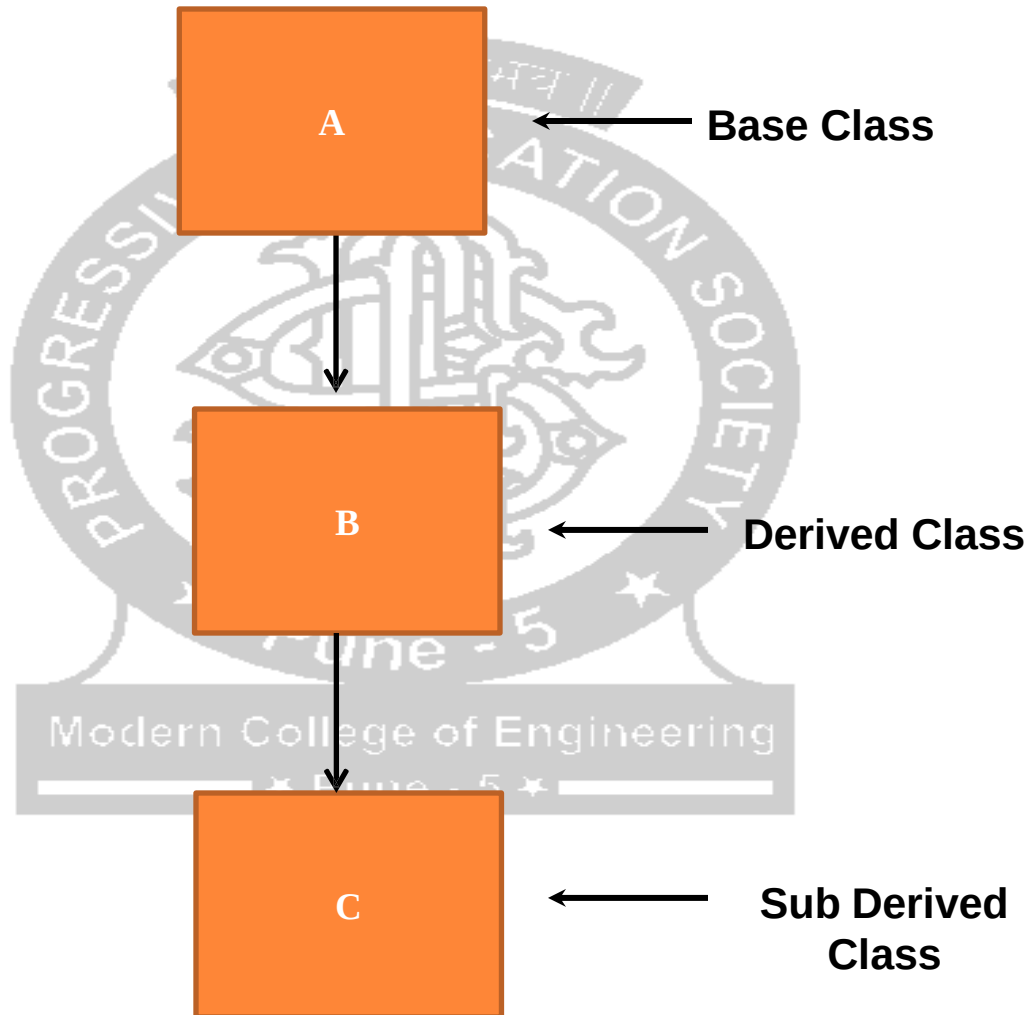
Note:

If constructor in super class is a parametrized constructor then it is mandatory to define constructor in child class.

Using key word we are using explicit call to constructor in parent class.



MULTILEVEL INHERITANCE



MULTILEVEL INHERITANCE

```
class Vehicle
{
Vehicle()
{
System.out.print("\n"+"class
vehicle");
}
}

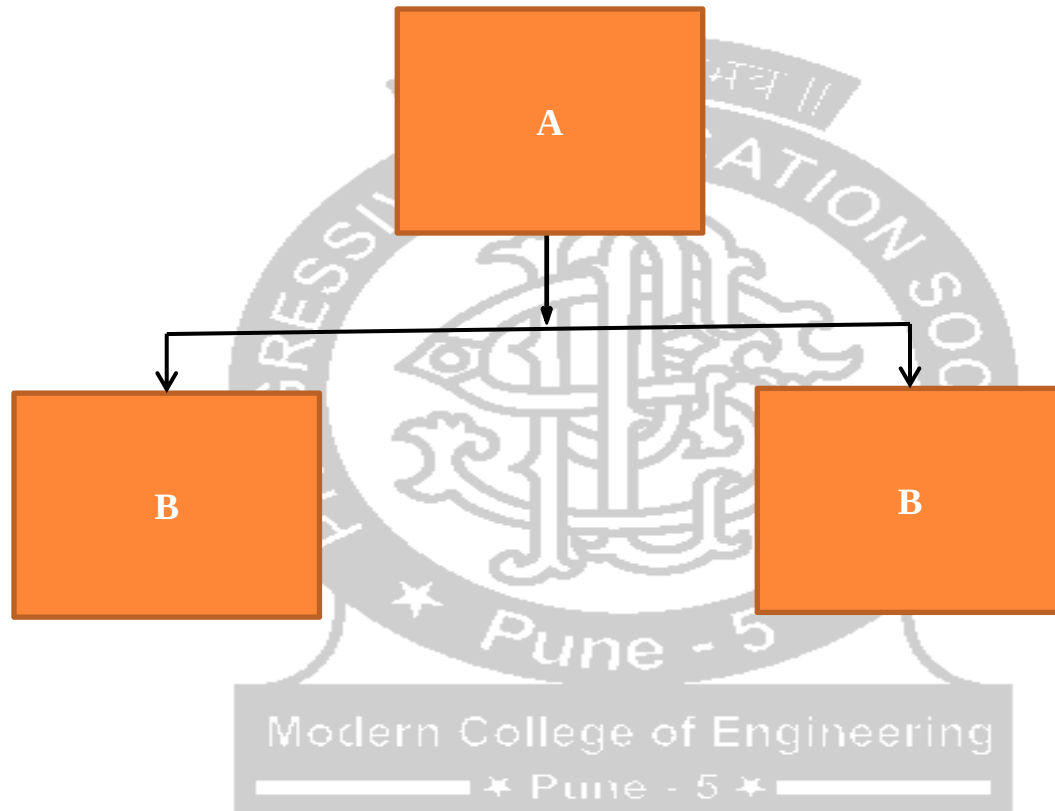
class Car extends Vehicle
{
Car()
{
System.out.print("\n"+"class
car");
}
}
```

```
class Maruti extends Car
{
Maruti()
{
System.out.print("\n"+"class
Maruti");
}
}

public class Multilevel_inheritance
{
public static void main(String[]
args) {
Maruti obj = new Maruti();
}
}
```



HIERARCHICAL INHERITANCE



HIERARCHICAL INHERITANCE

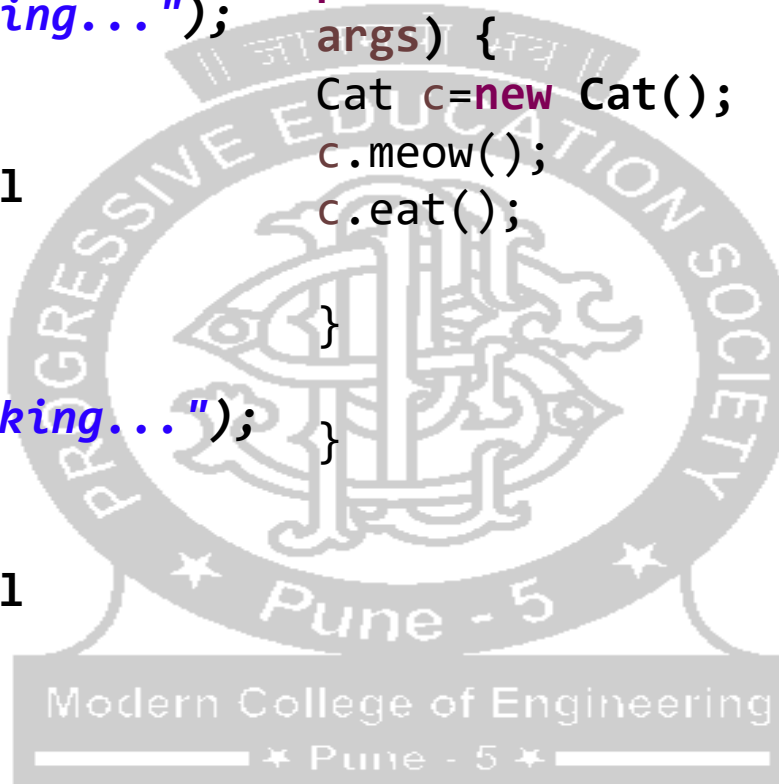
```
class Animal
{
void eat()
{
System.out.println("eating...");
}
}

class Dog extends Animal
{
void bark()
{
System.out.println("barking...");
}
}

class Cat extends Animal
{
void meow()
{
System.out.println("meowing...");
}
}

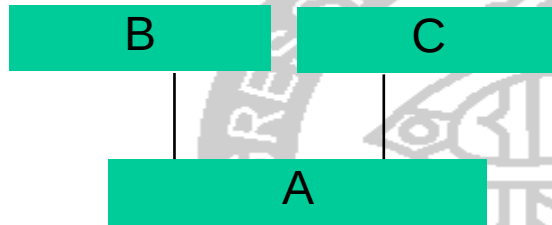
public class
Heirarchical_inheritance {

public static void main(String[]
args) {
Cat c=new Cat();
c.meow();
c.eat();
}
}
```



SUPPORTING MULTIPLE INHERITANCE IN JAVA

- Java supports single inheritance, multilevel inheritance and hierarchical inheritance.
- Java does not support multiple inheritance.



- There cannot be two super classes for a single subclass.
- To overcome this limitation interface was introduced in java.



JAVA INTERFACE

- ❖ Interface in Java is same as class.
- ❖ Like classes interface also has methods and constants.
- ❖ A Java *interface* is a collection of **constants** and **abstract methods**
 - abstract method: a method header without a method body; we declare an abstract method using the modifier abstract
 - since all methods in an interface are abstract, the abstract modifier is usually not written.
 - The variables are static and final(once initialised the value in variable cannot be changed).
- ❖ Methods and variables in an interface have public visibility by default



SYNTAX FOR DEFINING AN INTERFACE

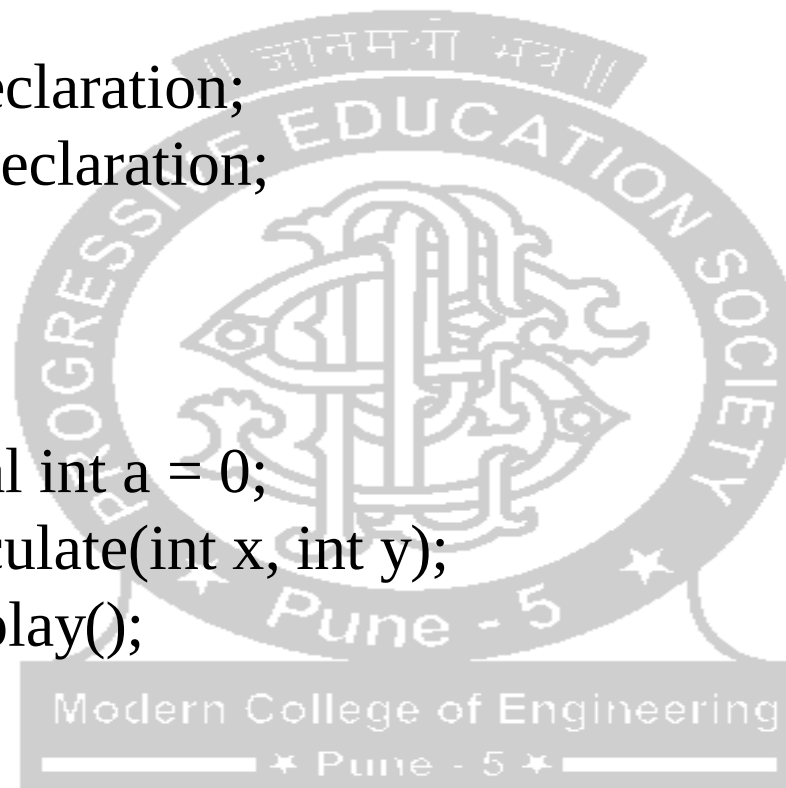
- ❖ The definition uses **interface** keyword.

```
interface name
{
    variable declaration;
    function declaration;
}
```

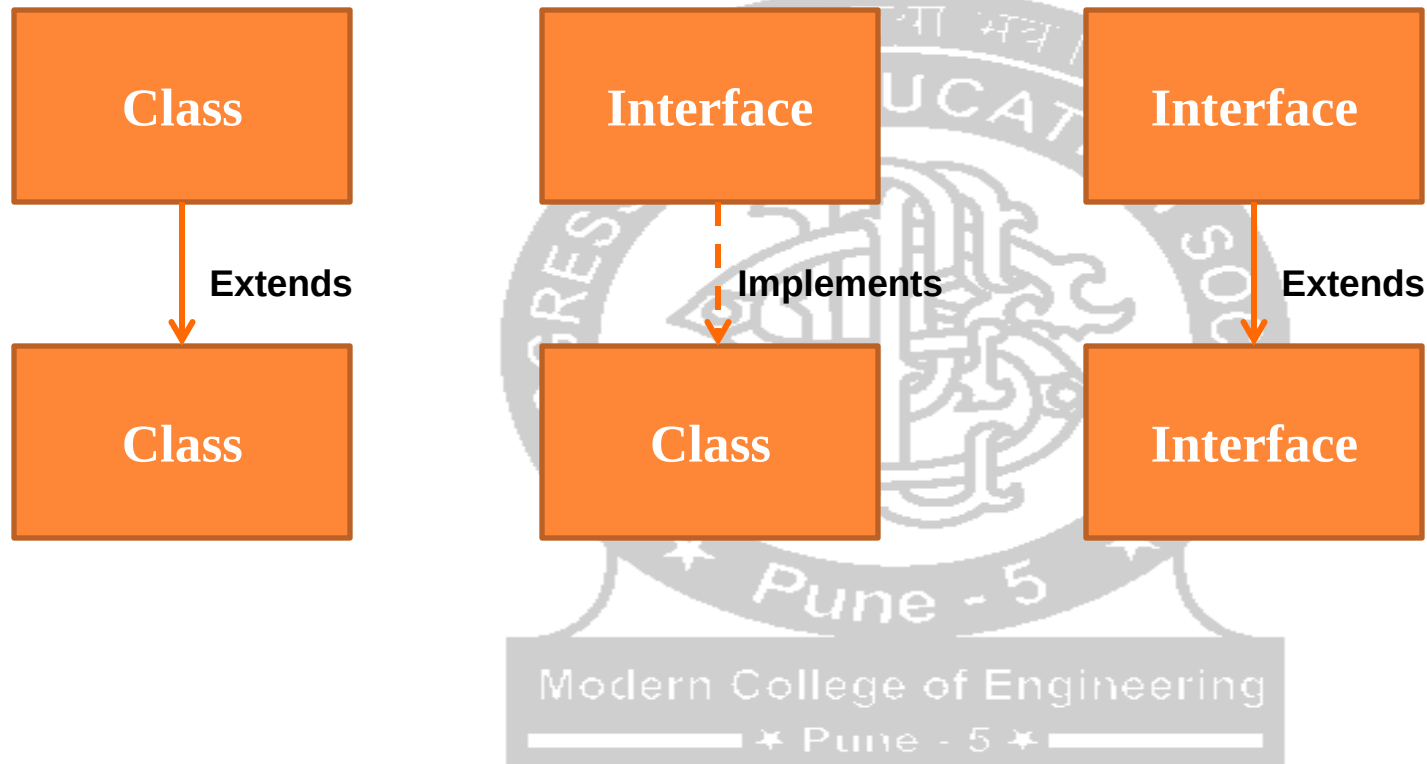
Ex:

```
interface data
{
    static final int a = 0;
    void calculate(int x, int y);
    void display();
}
```

**** we cannot define the methods in interface.**



RELATIONSHIP BETWEEN CLASSES AND INTERFACE



DIFFERENCE BETWEEN INTERFACE AND CLASS

Class	Interface
The keyword used to create a class is “class”	The keyword used to create an interface is “interface”
A class can be instantiated i.e, objects of a class can be created.	An Interface cannot be instantiated i.e, objects cannot be created.
Classes does not support multiple inheritance.	Interface supports multiple inheritance.
It can be inherit another class.	It cannot inherit a class.
It can be inherited by another class using the keyword ‘extends’.	It can be inherited by a class by using the keyword ‘implements’ and it can be inherited by an interface using the keyword ‘extends’.
It can contain constructors.	It cannot contain constructors.
It contain abstract or non abstract methods.	It contains abstract methods only.
Variables and methods in a class can be declared using any access specifier (public, private, default, protected)	All variables and methods in a interface are declared as public.
Variables in a class can be static, final or neither.	All variables are static and final.



IMPLEMENTATION OF INTERFACE

interface data

```
{  
    static final int a = 10;  
    void calculate(int x, int y);  
    void display();  
}
```

Interface declaration

public class A implements data {

public void display()

```
{  
    System.out.println("Display method implementation in Class A");  
}
```

public void calculate(int x, int y)

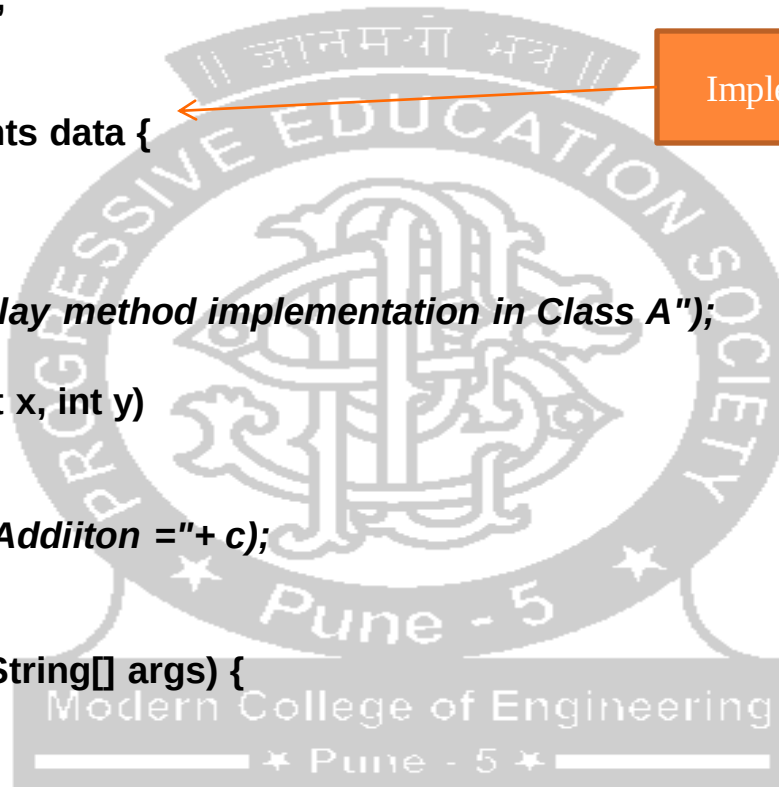
```
{  
    int c = x+y;  
    System.out.println("Addiiton =" + c);  
}
```

public static void main(String[] args) {

```
A obj = new A();  
obj.display();  
obj.calculate( 2, a );
```

```
}  
}
```

Implementation of interface



EXTENDING INTERFACE

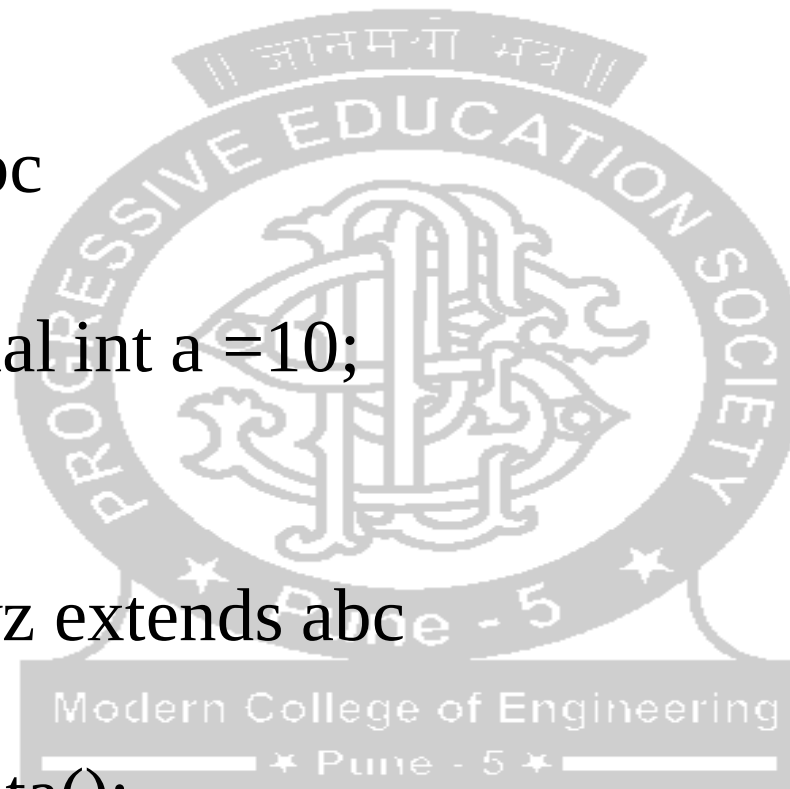
- Interfaces can be extended as classes using keyword extends.

Ex: interface abc

```
{  
    static final int a = 10;  
}
```

interface xyz extends abc

```
{  
    void getdata();  
    void show();  
}
```



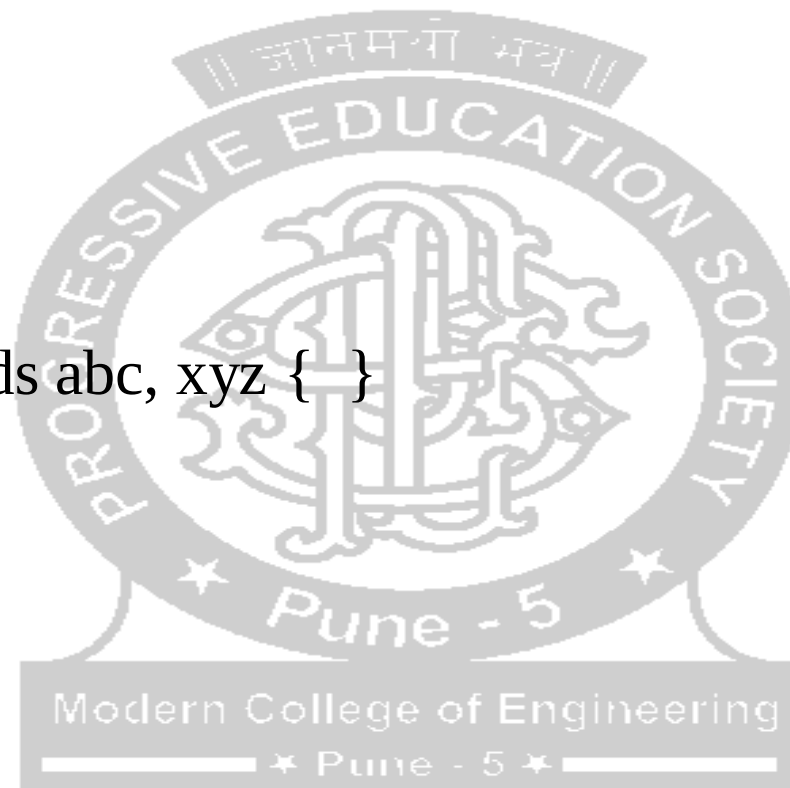
MULTIPLE INTERFACES

- We can also combine several interfaces together into a single interface

```
Interface abc { }
```

```
Interface xyz { }
```

```
Interface pqr extends abc, xyz { }
```



IMPLEMENT AND EXTEND KEYWORDS

```
interface i1 { }
```

```
interface i2 { }
```

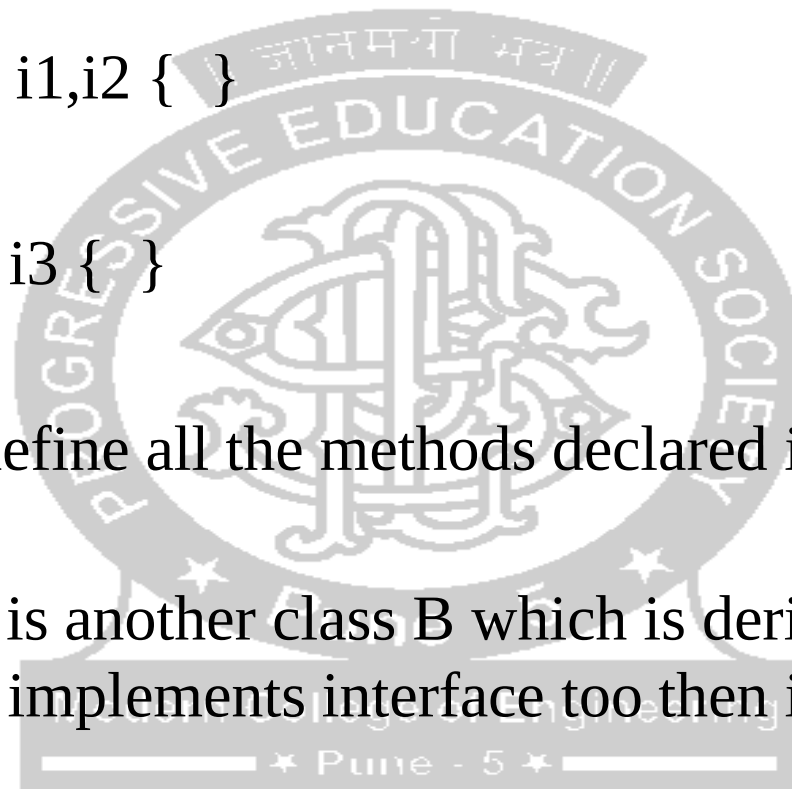
```
interface i3 extends i1,i2 { }
```

```
class A implements i3 { }
```

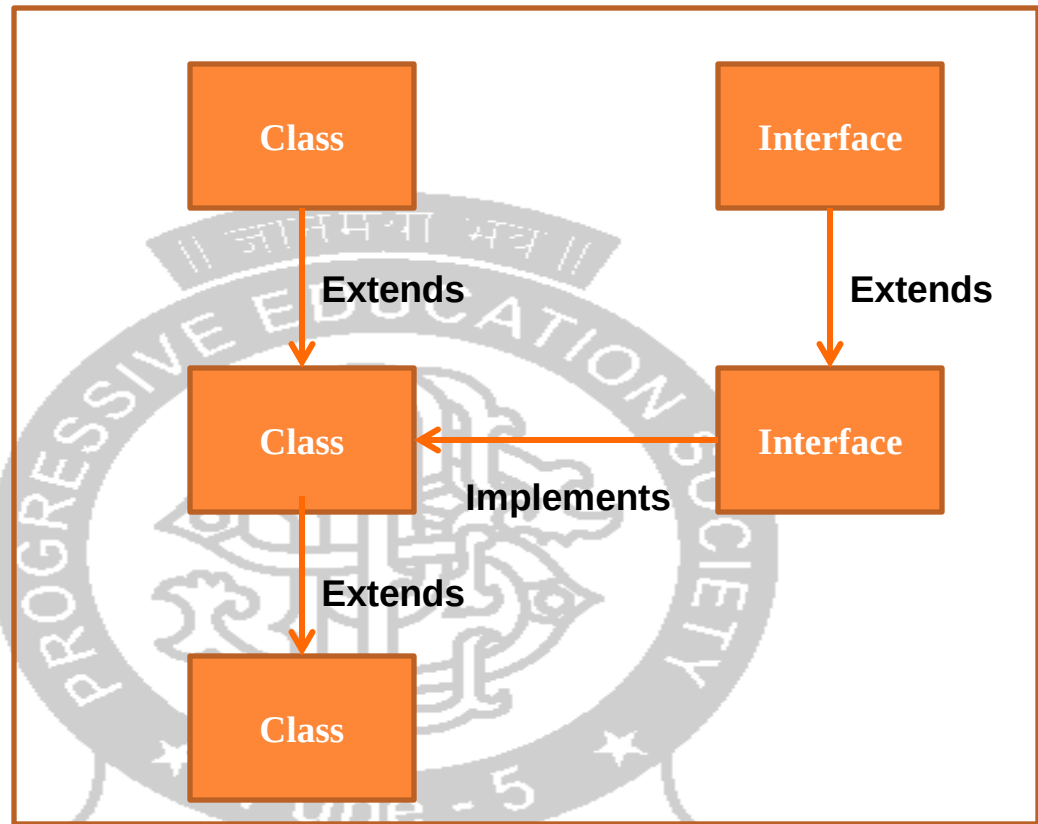
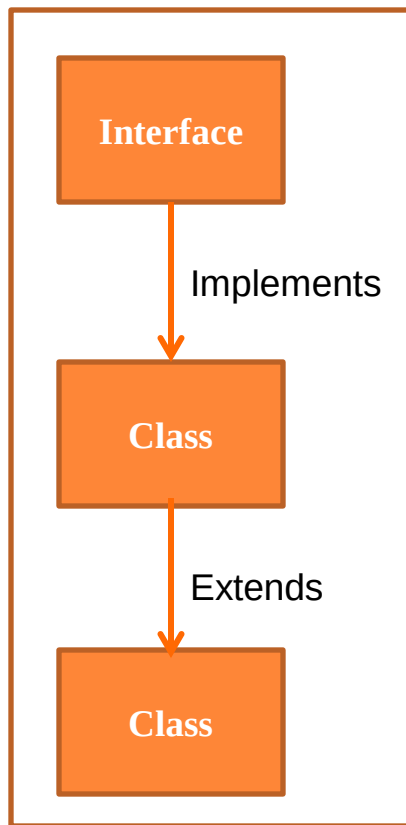
Its class A duty to define all the methods declared in i1,i2 and i3.

Now suppose there is another class B which is derived from class A which implements interface too then it is possible how?

```
Class B extends A implements i3 { }
```



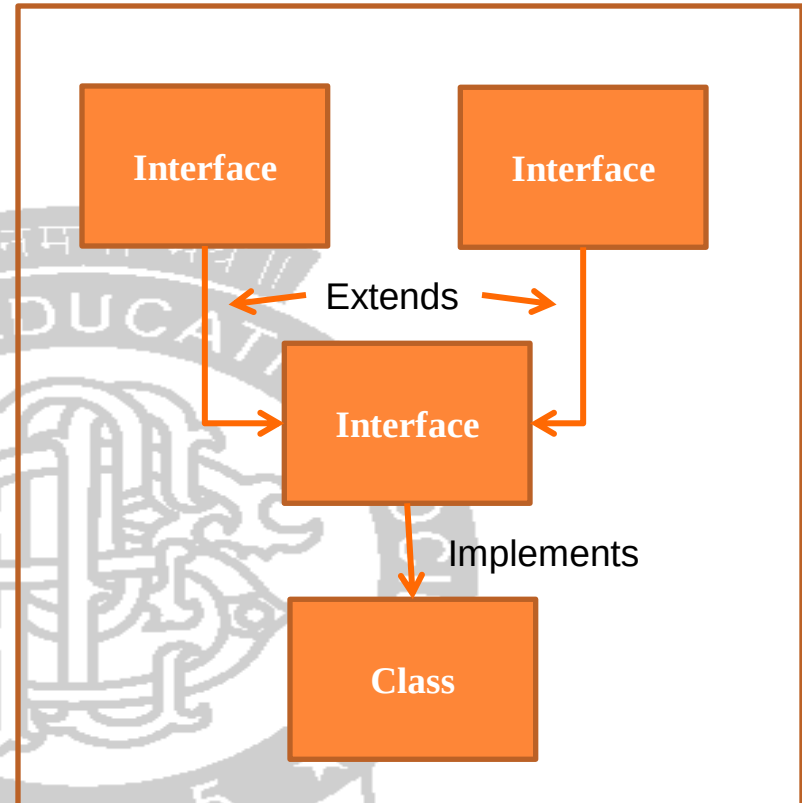
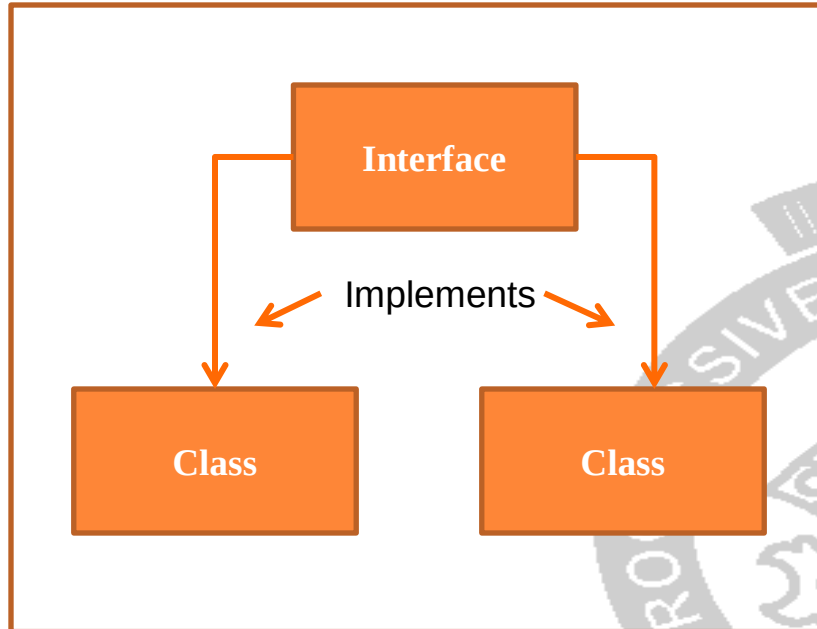
IMPLEMENTING INTERFACES

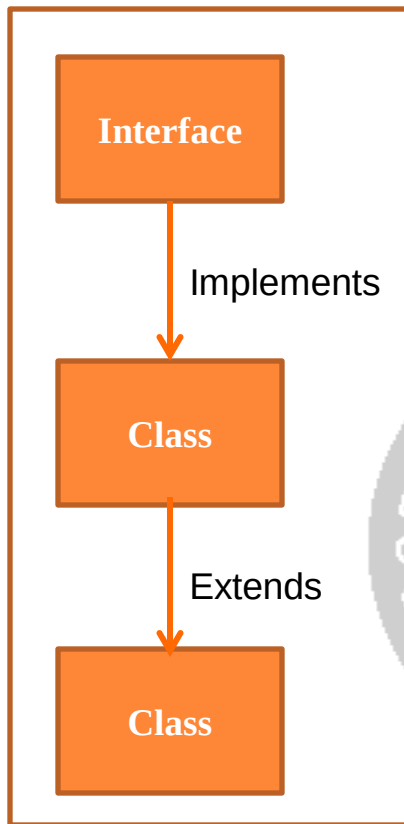


Modern College of Engineering

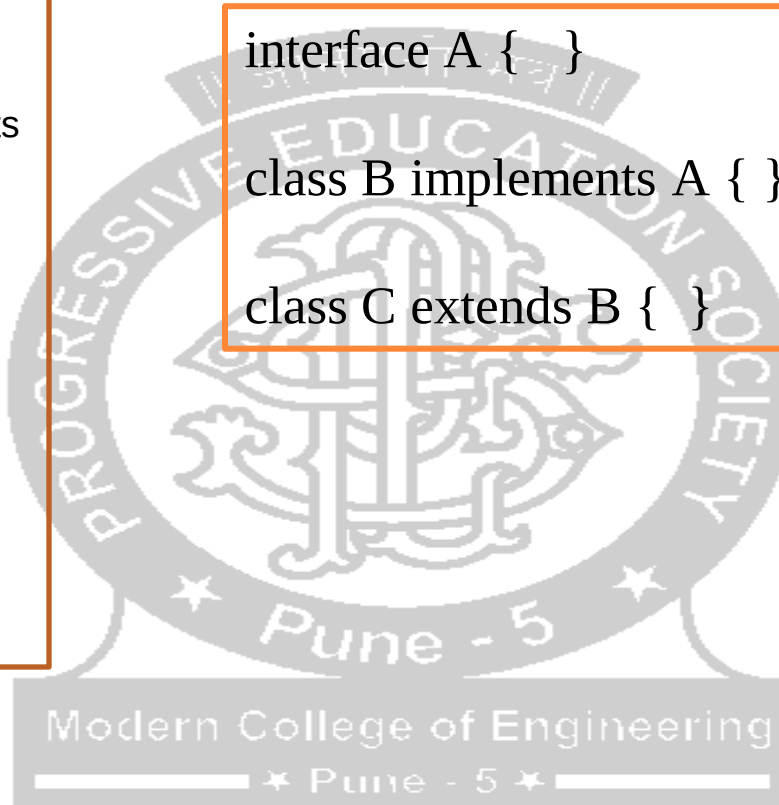
★ Pune - 5 ★

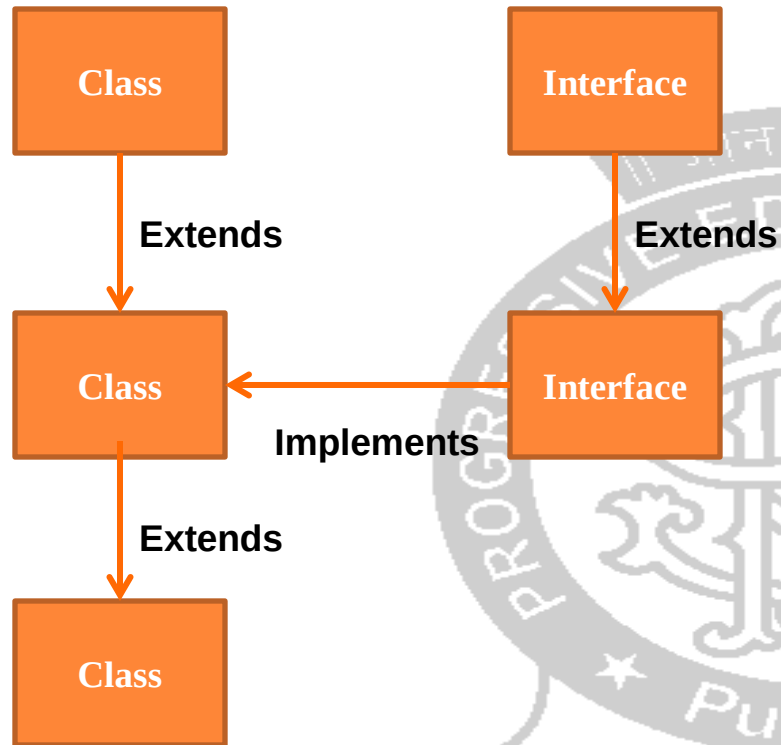
IMPLEMENTING INTERFACES





```
interface A { }  
class B implements A { }  
class C extends B { }
```





```
interface I1 { }
```

```
interface I2 extends I1 { }
```

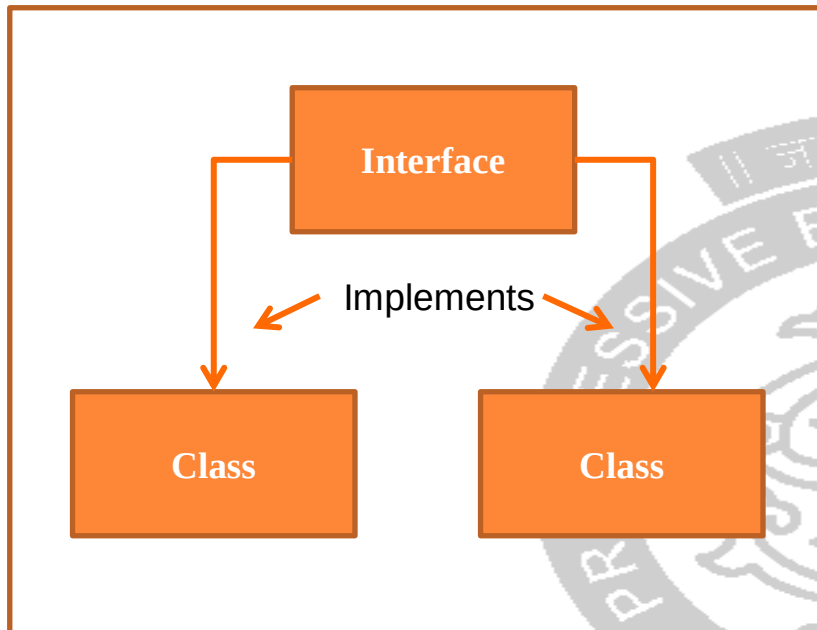
```
class A { }
```

```
class B extends A implements I2{ }
```

```
class c extends B { }
```

Modern College of Engineering

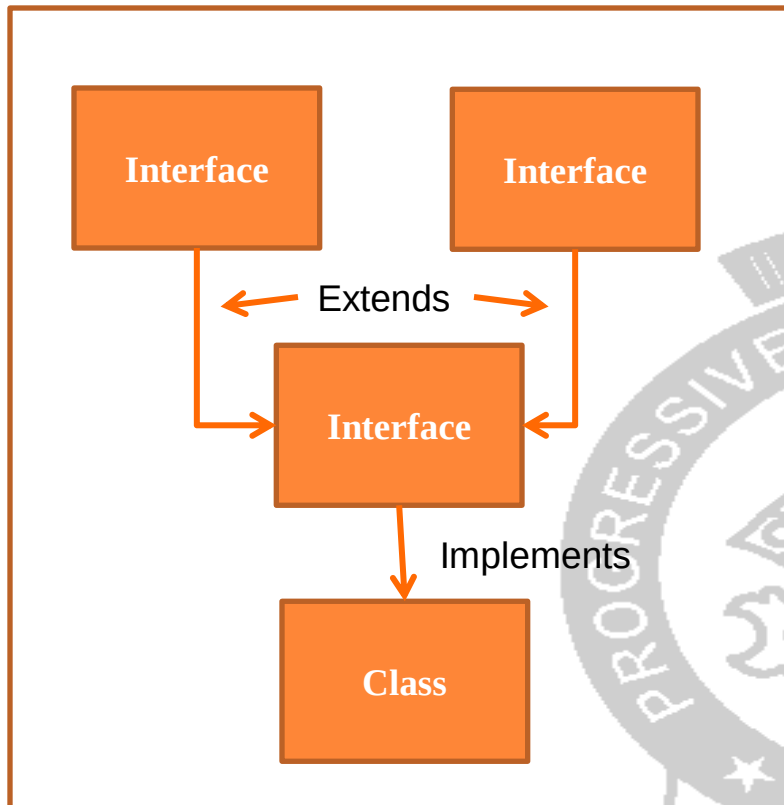
★ Pune - 5 ★



```
interface A { }
```

```
class B implements A { }
```

```
class C implements A { }
```



```
interface A { }  
interface B { }  
interface C extends A,B  
{ }  
class D implements C  
{ }
```

POLYMORPHISM

There are two types of polymorphism in java:

- 1) **Static Polymorphism** also known as compile time polymorphism.
- 2) **Dynamic Polymorphism** also known as runtime polymorphism.

Compile time Polymorphism (or Static polymorphism)

Polymorphism that is determined during compile time is known as static polymorphism.

Method overloading and operator overloading is an example of compile time polymorphism.

Ex:

- ❖ Method overloading is one of the way java supports static polymorphism.
- ❖ Here we have two definitions of the same method add() which add method would be called is determined by the parameter list at the compile time.

ex:

```
add(int a, int b)
```

```
add(int a, int b, int c)
```

- ❖ That is the reason this is also known as compile time polymorphism.

Runtime Polymorphism (Dynamic polymorphism)

- ❖ Method overriding is an example of run time polymorphism.
- ❖ Runtime polymorphism is also known as dynamic method dispatch



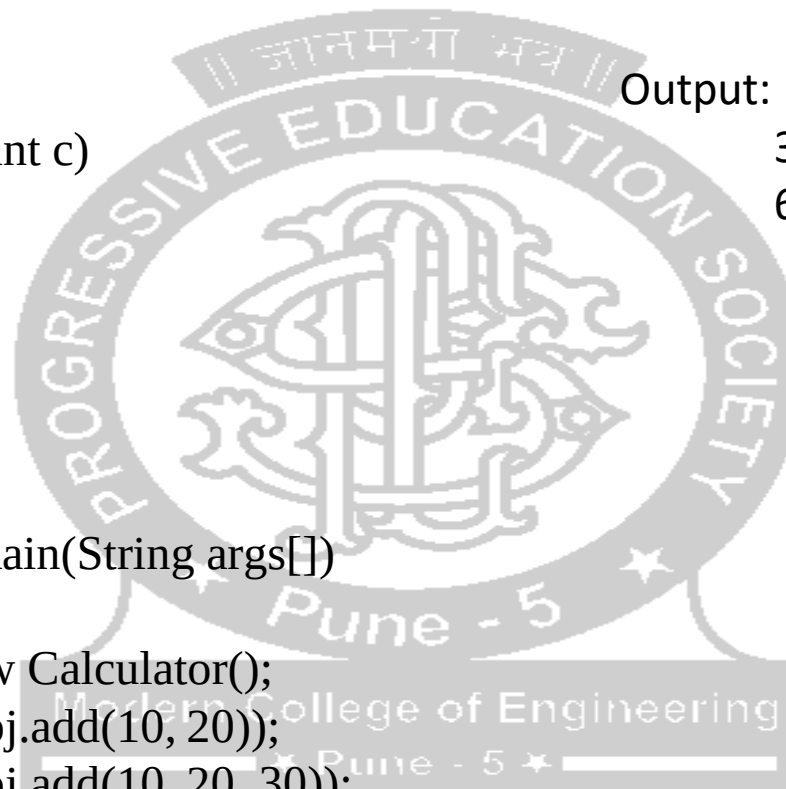
COMPILE TIME POLYMORPHISM

```
class Calculator
{
int add(int a, int b)
{
return a+b;
}
int add(int a, int b, int c)
{
return a+b+c;
}
}

public class Demo
{
public static void main(String args[])
{
Calculator obj = new Calculator();
System.out.print(obj.add(10, 20));
System.out.print(obj.add(10, 20, 30));
}
}
```

Output:

30
60



METHOD OVERRIDING

Declaring a method in sub class which is already present in parent class is known as method overriding

Method overriding is used so that child class can perform its own implementation of a method (function) which is provided in parent class.

Method in parent class is called overridden method and the method in child class is called overriding method.

Ex:

We have an Animal class which implements method eat();

We have a child class called as lion and another child class called as goat. Both the child classes eat different things so they can implement their own implementation.

```
class Animal
```

```
{  
    void eat()  
{  
    System.out.print("Animals eat anything");  
}}  
class Lion extends Animal
```

```
{  
    void eat()  
{  
    System.out.print("Lion eats flesh");  
}}  
}
```

```
class Goat extends Animal
```

```
{  
    void eat()  
{  
    System.out.print("Goat eats grass");  
}}  
}
```

```
class Data
```

```
{  
    public static void main(String args)  
{  
        Goat obj = new Goat();  
    }  
}
```



DYNAMIC METHOD DISPATCH

```
class Animal
{
    public void move()
    {
        System.out.println("Animals can move");
    }
}

class Dog extends Animal
{
    public void move()
    {
        System.out.println("D
ogs can walk and run");
    }
}

public class TestDog {
    public static void main(String args[]) {
        Animal a = new Animal();
        Dog b = new Dog();
        a.move();
        b.move();
    }
}
```

In the example move() method is an overridden method on parent class and overriding method in child class.

The output of the program is:

Animals can move

Dogs can walk and run

This is because we have separate object for parent and child class.

The object if created of child class will not be able to call the overridden method of parent class. Thus, we learn dynamic method dispatch

// Animal reference and object

// Dog reference and object

// runs the method in Animal class

// runs the method in Dog class

DYNAMIC METHOD DISPATCH

```
class Animal
```

```
{
```

```
void move()
```

```
{
```

```
System.out.print("Animals can  
move");
```

```
}
```

```
}
```

```
class Dog extends Animal
```

```
{
```

```
void move()
```

```
{
```

```
System.out.print("Dogs can walk and  
run");
```

```
}
```

```
}
```

```
public class Dynamic_dispatch {
```

```
public static void main(String[]  
args) {
```

```
Animal a = new Animal();
```

```
// Animal reference and object.
```

```
// This would call the move()
```

```
method of parent class Animal
```

```
Dog d = new Dog();
```

```
// Dog reference and object.
```

```
// This would call the move()
```

```
method of child class Dog
```

```
Animal b = new Dog();
```

```
// Animal reference but Dog object
```

```
// This would call the move()
```

```
method of child class Dog
```

```
d.move();
```

```
a.move();
```

```
// runs the method in Animal class
```

```
b.move();
```

```
// runs the method in Dog class
```

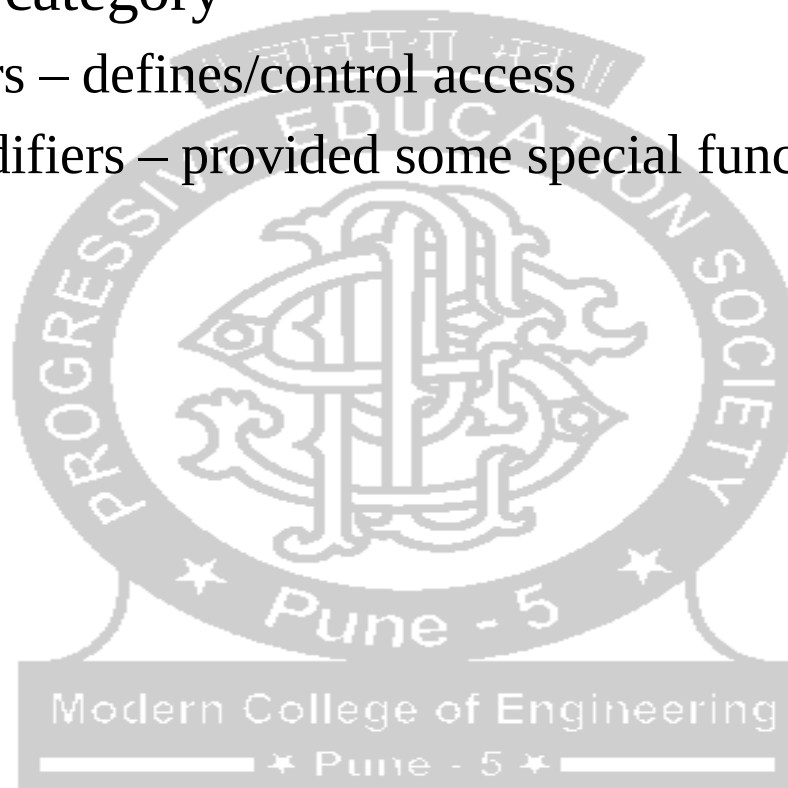
```
}
```

```
}
```



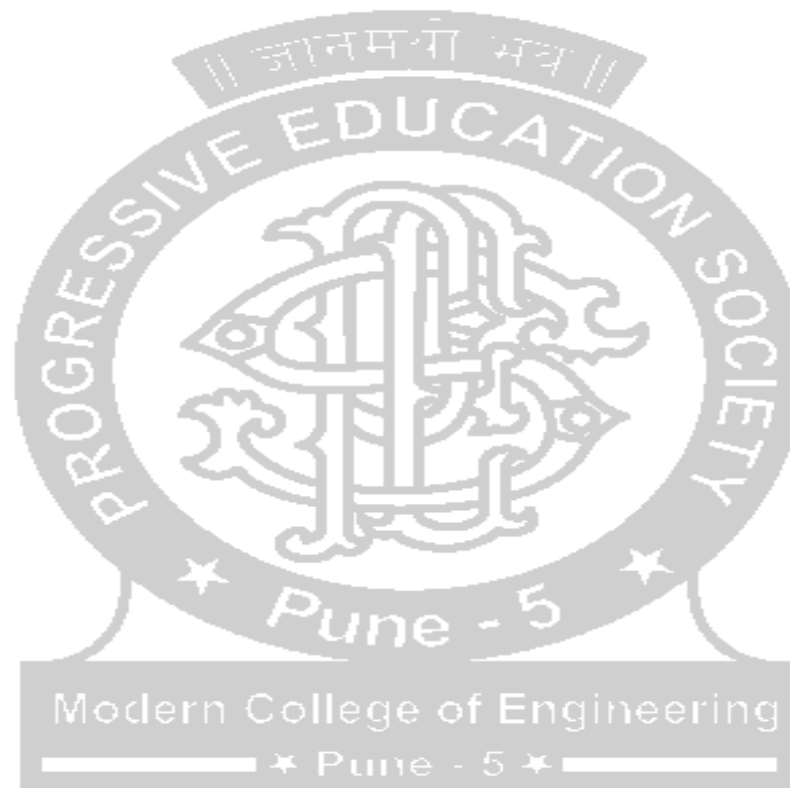
MODIFIERS AVAILABLE IN JAVA

- Divided into two category
 - Access Modifiers – defines/control access
 - Non-access Modifiers – provided some special functionality



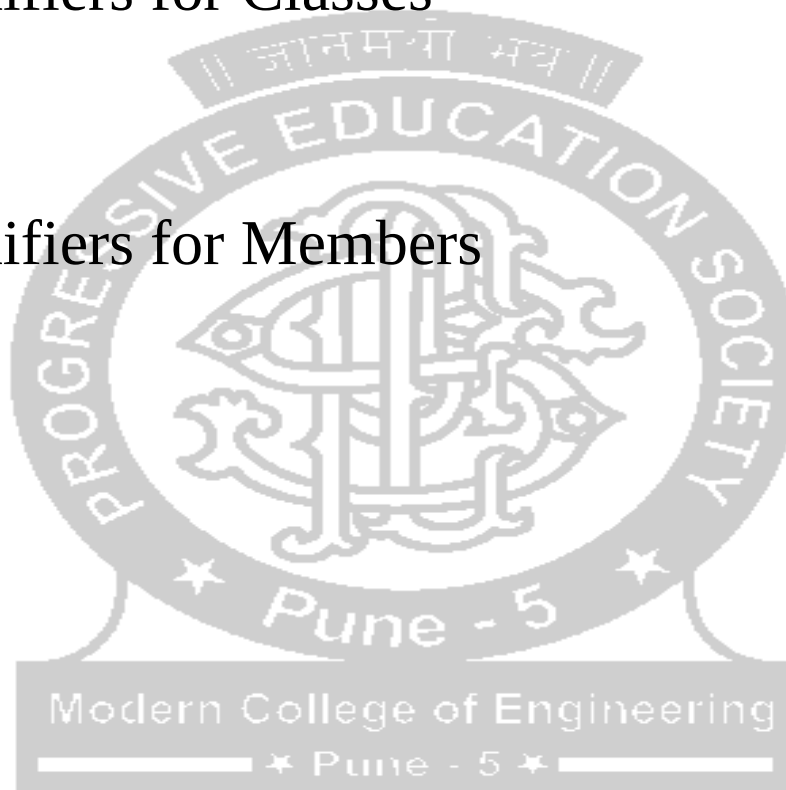
ACCESS MODIFIERS

- Public
- Private
- Protected
- Default



NON-ACCESS MODIFIERS

- Non-Access Modifiers for Classes
 - Final
 - Abstract
- Non-Access Modifiers for Members
 - Static
 - Final
 - Abstract
 - Transient
 - Synchronized
 - Volatile
 - Native



STATIC VARIABLE AND METHODS.

The **static keyword** in Java is used for memory management mainly.

We can apply static keyword with variables, methods, blocks and nested classes.

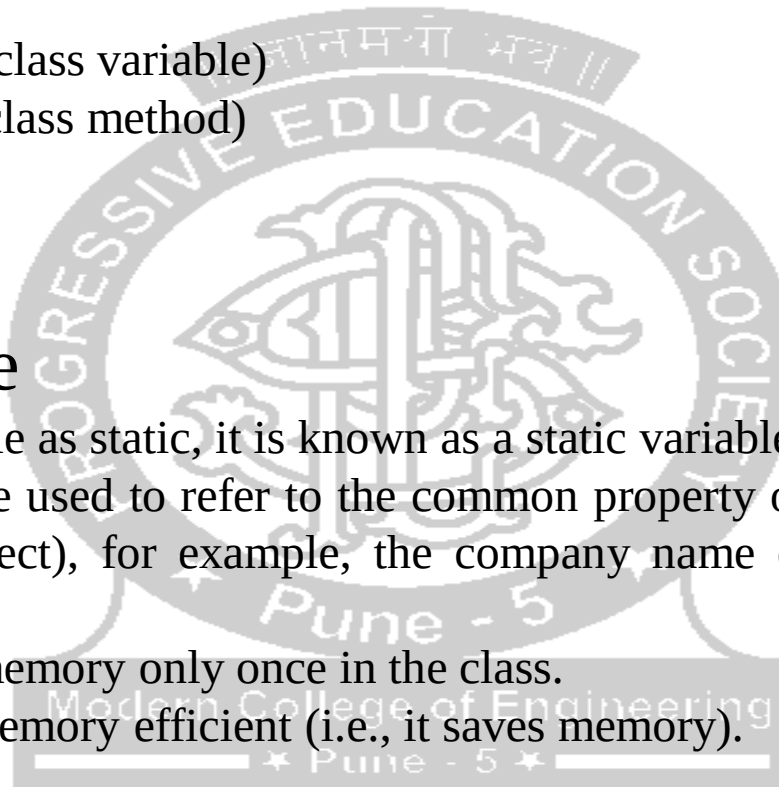
The static keyword belongs to the class than an instance (object) of the class.

The static can be:

1. Variable (also known as a class variable)
2. Method (also known as a class method)
3. Block
4. Nested class

Java static variable

- ❖ If you declare any variable as static, it is known as a static variable.
- ❖ The static variable can be used to refer to the common property of all objects (which is not unique for each object), for example, the company name of employees, college name of students, etc.
- ❖ The static variable gets memory only once in the class.
- ❖ It makes your program memory efficient (i.e., it saves memory).



JAVA STATIC VARIABLE

Ex:

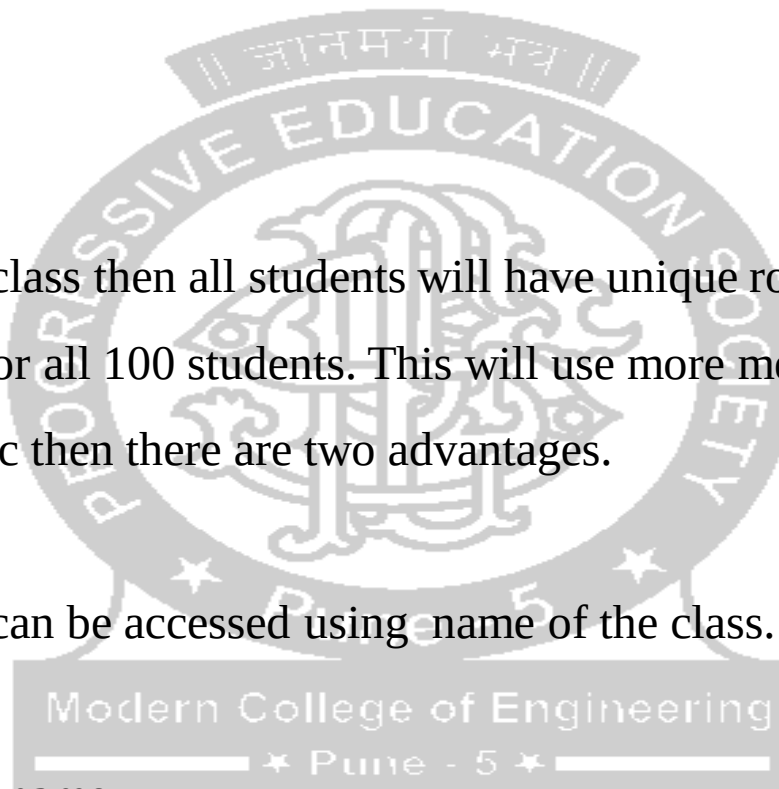
```
class Student
{
    int roll;
    String name;
    String college_name;
}
```

Here if I have 100 students in class then all students will have unique roll number and name but college name would be same for all 100 students. This will use more memory. But, if we declare college_name variable as static then there are two advantages.

1. Memory will be saved.
2. The college_name variable can be accessed using name of the class.

Syntax to use static variable:

class name. static variable name.



JAVA STATIC VARIABLE

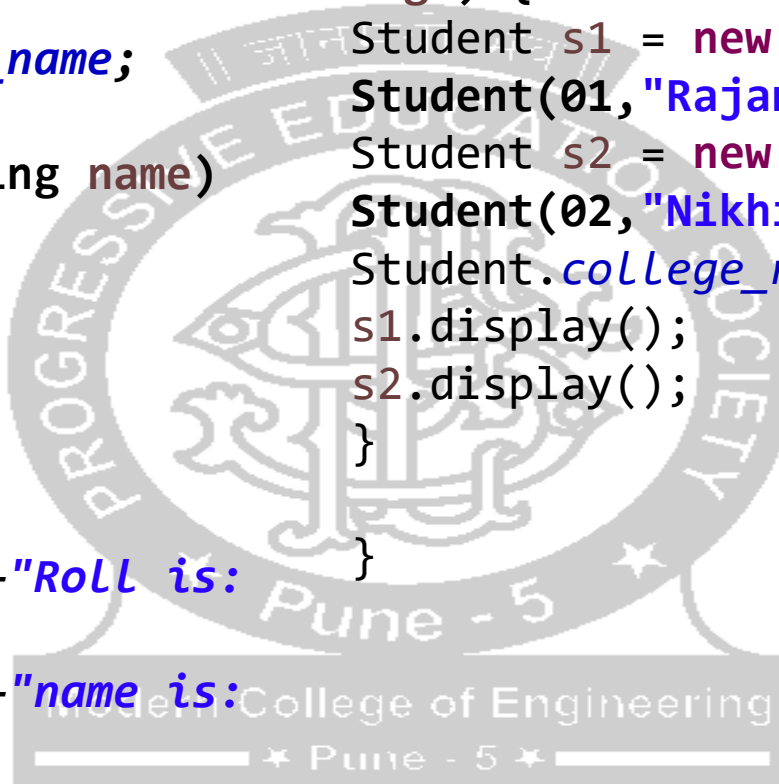
```
class Student
{
    int roll;
    String name;
    static String college_name;

    Student(int roll, String name)
    {
        this.roll = roll;
        this.name = name;
    }

    void display()
    {
        System.out.print("\n"+"Roll is:
        "+roll);
        System.out.print("\n"+"name is:
        "+name);
        System.out.print("\n"+"college name
        is: "+college_name);
    }
}
```

```
public class Static_variable {

    public static void main(String[]
    args) {
        Student s1 = new
        Student(01,"Rajan");
        Student s2 = new
        Student(02,"Nikhil");
        Student.college_name = "MCOE";
        s1.display();
        s2.display();
    }
}
```



STATIC METHODS

If you apply static keyword with any method, it is known as static method.

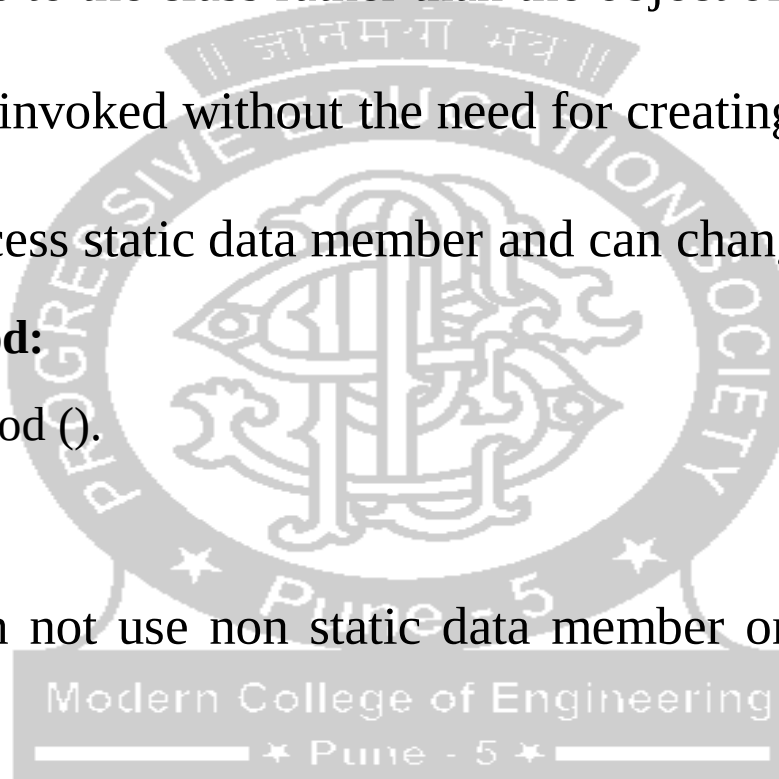
- A static method belongs to the class rather than the object of a class.
- A static method can be invoked without the need for creating an instance of a class.
- A static method can access static data member and can change the value of it.

Syntax to use static method:

class name. static method ().

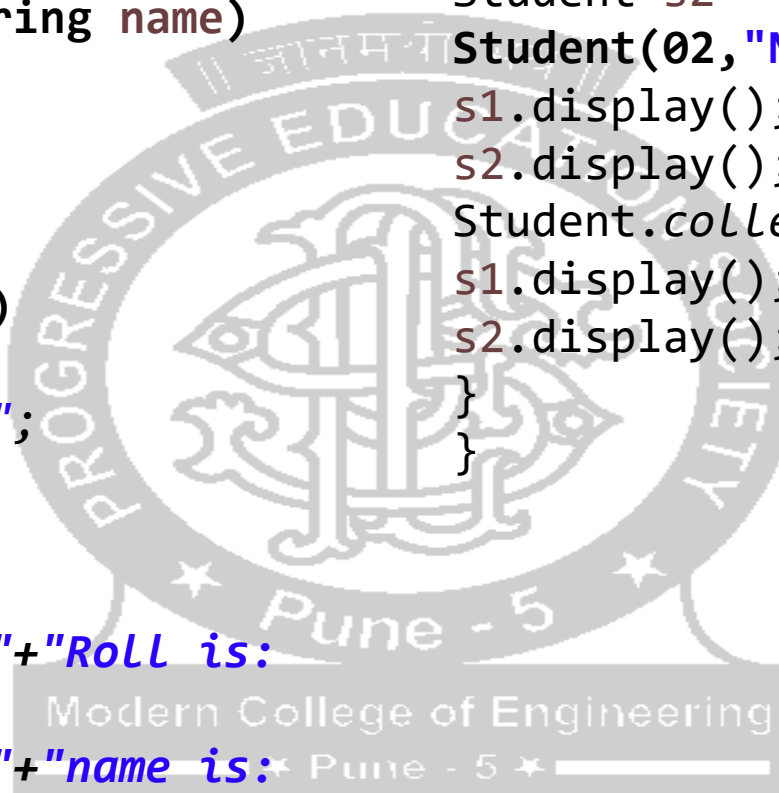
Rules of Static Method:

- ⑩ The static method can not use non static data member or call non-static method directly.
- ⑩ this and super cannot be used in static context.




```
class Student
{
    int roll;
    String name;
    static String college_name = "Modern
    College of Engineering";
    Student(int roll, String name)
    {
        this.roll = roll;
        this.name = name;
    }
    static void college()
    {
        college_name = "MCOE";
    }
    void display()
    {
        System.out.print("\n"+"Roll is:
        "+roll);
        System.out.print("\n"+"name is:
        "+name);
        System.out.print("\n"+"college name
        is: "+college_name);
    }
}
```

```
public class Static_variable
{
    public static void main(String[]
    args) {
        Student s1 = new
        Student(01,"Rajan");
        Student s2 = new
        Student(02,"Nikhil");
        s1.display();
        s2.display();
        Student.college();
        s1.display();
        s2.display();
    }
}
```



ABSTRACT METHODS AND CLASSES.

A class that is declared using “**abstract**” keyword is known as abstract class.

Why you need an Abstract class. Consider an example:

Normal or concrete class

```
class Animal
{
    public void sound()
    {
        System.out.print(" some sound");
    }
}

class Lion extends Animal
{
    public void sound()
    {
        System.out.print(" roar");
    }
}

class Dog extends Animal
{
    public void sound()
    {
        System.out.print(" bark");
    }
}
```

Every child class has its own implementation. Thus it is of no use in defining sound function in parent class

Abstract class

```
abstract class Animal
{
    public abstract void sound();
}

class Lion extends Animal
{
    public void sound()
    {
        System.out.print(" roar");
    }
}

class Dog extends Animal
{
    public void sound()
    {
        System.out.print(" bark");
    }
}
```

An abstract method in abstract class has to be implemented by all the subclasses of Animal class

ABSTRACT METHODS AND CLASSES.

- ❖ A class that is declared using “**abstract**” keyword is known as abstract class.
- ❖ It can have abstract methods(methods without body) as well as concrete methods (regular methods with body).
- ❖ A normal class(non-abstract class) cannot have abstract methods.
- ❖ Abstract class cannot be instantiated means that you cannot create an object of that class as it is not a complete class

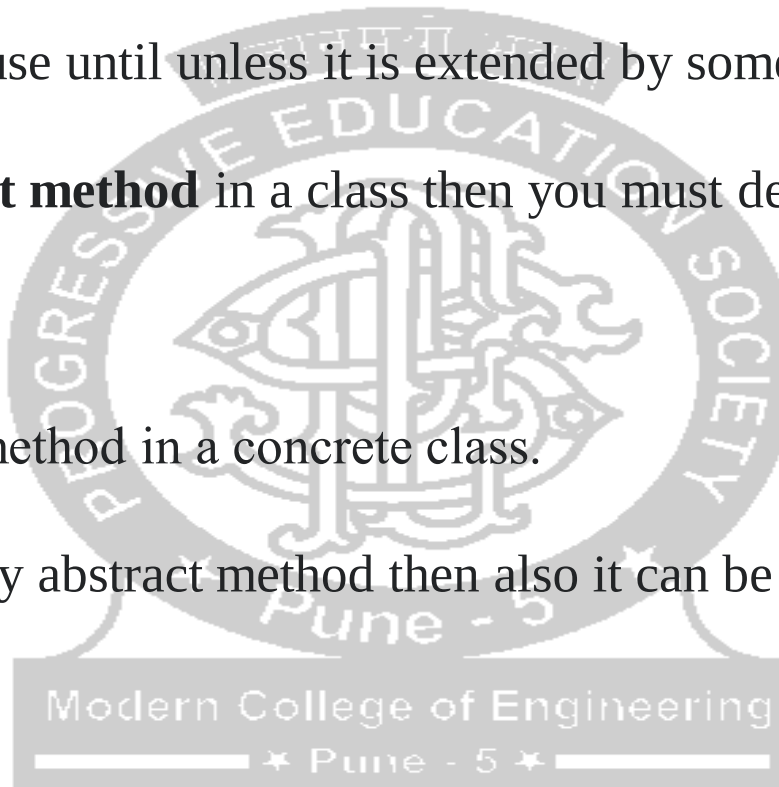
Syntax:

Declaration using abstract keyword

```
abstract class A  
{  
  abstract void myMethod(); //This is abstract method  
  //This is concrete method with body  
  void anotherMethod()  
  {  
    //Does something  
  }  
}
```

NOTE: ABSTRACT METHODS AND CLASSES.

- ❖ If a child does not implement all the abstract methods of abstract parent class, then the child class must need to be declared abstract as well.
- ❖ An abstract class has no use until unless it is extended by some other class.
- ❖ If you declare an **abstract method** in a class then you must declare the class abstract as well.
- ❖ You can't have abstract method in a concrete class.
- ❖ If a class is not having any abstract method then also it can be marked as abstract.



```

abstract class Data
{
    abstract public void
    calculation();
}
class Addition extends Data
{
    public void calculation()
    {
        int x=10,y=12;
        int c = x+y;
        System.out.print("result of child
        class Addition is: "+c);
    }
}
class Subtraction extends Data
{
    public void calculation()
    {
        int x=12,y=10;
        int c = x-y;
        System.out.print("result of child
        class Addition is: "+c);
    }
}

```

```

public class Abstarct_class {

    public static void main(String[]
    args) {
        // TODO Auto-generated method stub
        Addition obj1 = new Addition();
        Subtraction obj = new
        Subtraction();

        obj1.calculation();
        obj.calculation();
    }
}

```

Output:

result of child class Addition is: 22
 result of child class Addition is: 2



FINAL VARIABLES

- ✓ A variable can be declared as **final**. Doing so prevents its contents from being modified.
- ✓ This means that you must initialize a **final** variable when it is declared. For example:

```
final int FILE_NEW=1;  
final int FILE_OPEN=2;  
final int FILE_SAVE=3;  
final int FILE_SAVEAS=  
4; final int FILE_QUIT=  
5;
```

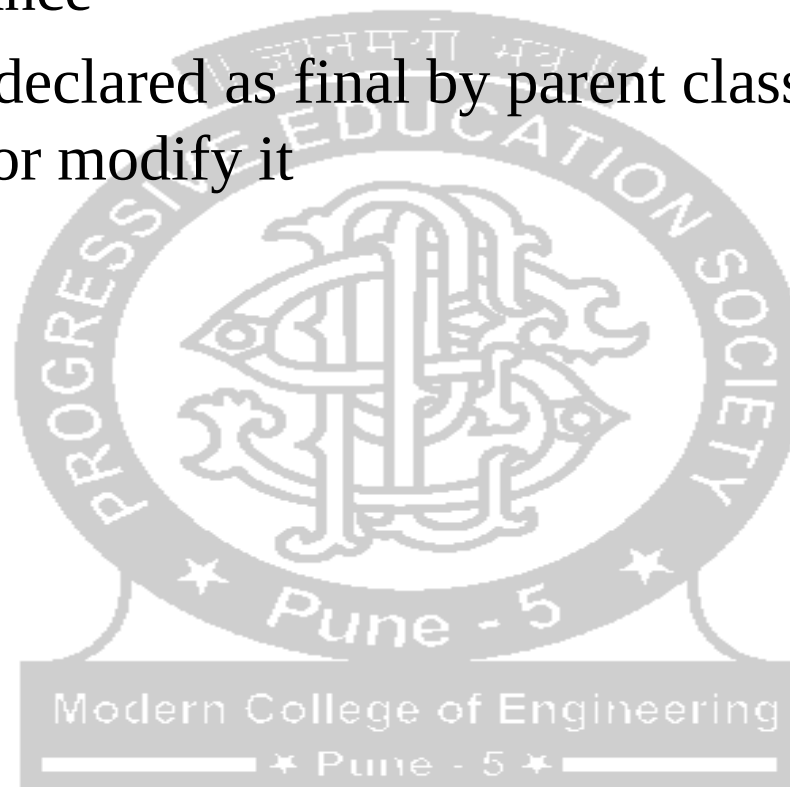
- ✓ Subsequent parts of your program can now use **FILE_OPEN**, etc., as if they were constants, without fear that a value has been changed.

- ✓ It is a common coding convention to choose all uppercase identifiers for **final** variables.
- ✓ Variables declared as **final** do not occupy memory on a per-instance basis.
- ✓ Thus, a **final** variable is essentially a constant.
- ✓ The keyword **final** can also be applied to methods, but its meaning is substantially different than when it is applied to variables.



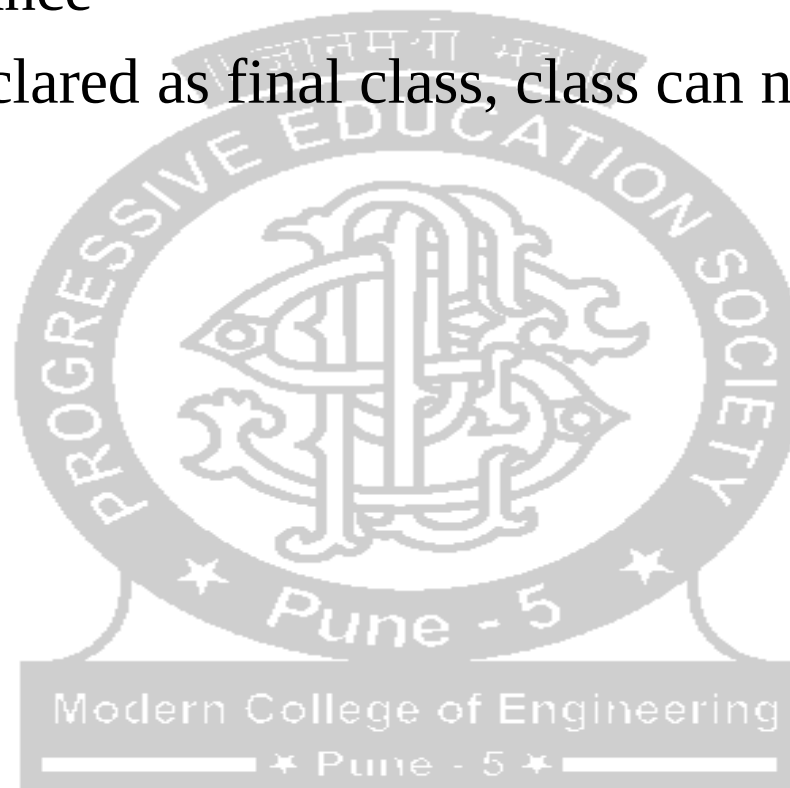
FINAL METHODS

- Useful in inheritance
- When method is declared as final by parent class, child can not override or modify it



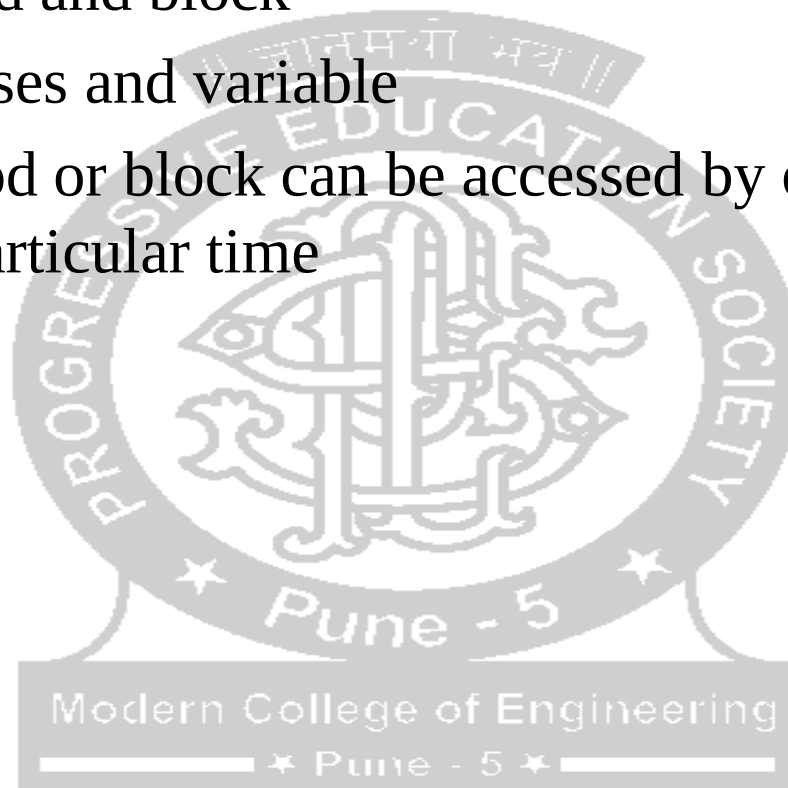
FINAL CLASS

- Useful in inheritance
- When class is declared as final class, class can not be extended



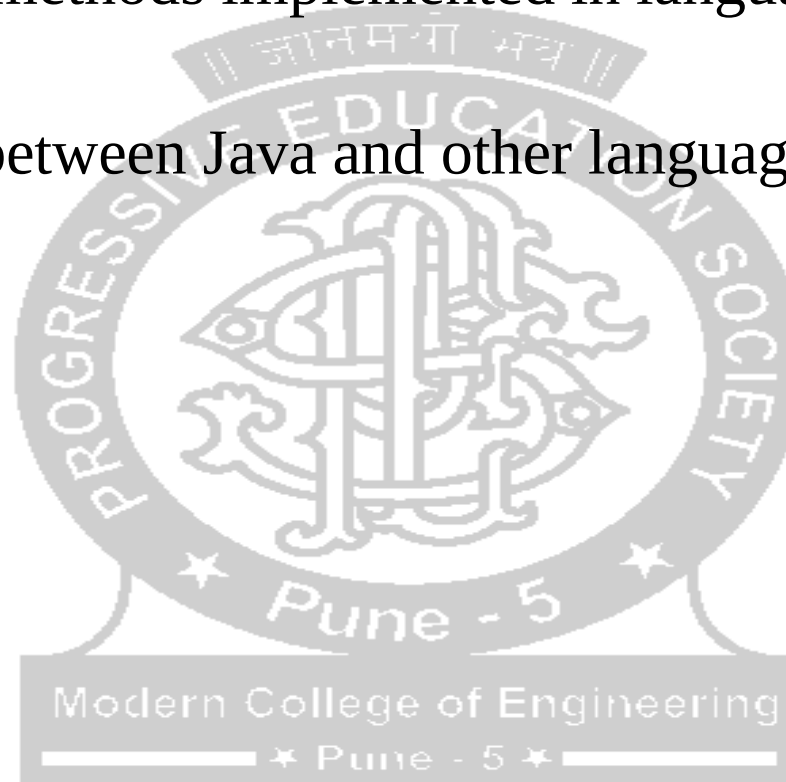
SYNCHRONIZED MODIFIERS

- Useful for method and block
- Not used for classes and variable
- Indicates a method or block can be accessed by only one thread at a one particular time



NATIVE MODIFIER

- Useful to access methods implemented in languages other than Java
- Act as interface between Java and other languages

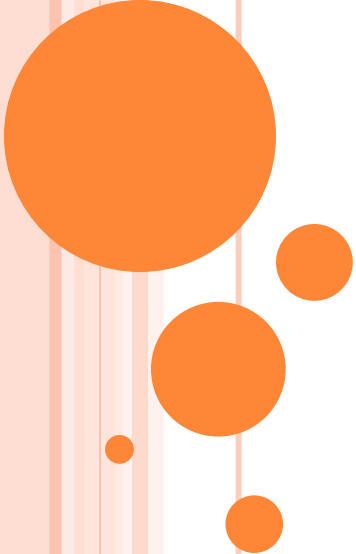


VOLATILE MODIFIER

- Useful in the case of Multithreading
- E.g. `public volatile int x;`
- Inform JVM that variable can be accessed and modified by multiple thread
- Changes made by one thread in this variable is immediately visible to other threads
- The volatile keyword does not cache the value of the variable and always read the variable from the main memory.

Modern College of Engineering

★ Pune - 5 ★



Thank you..